

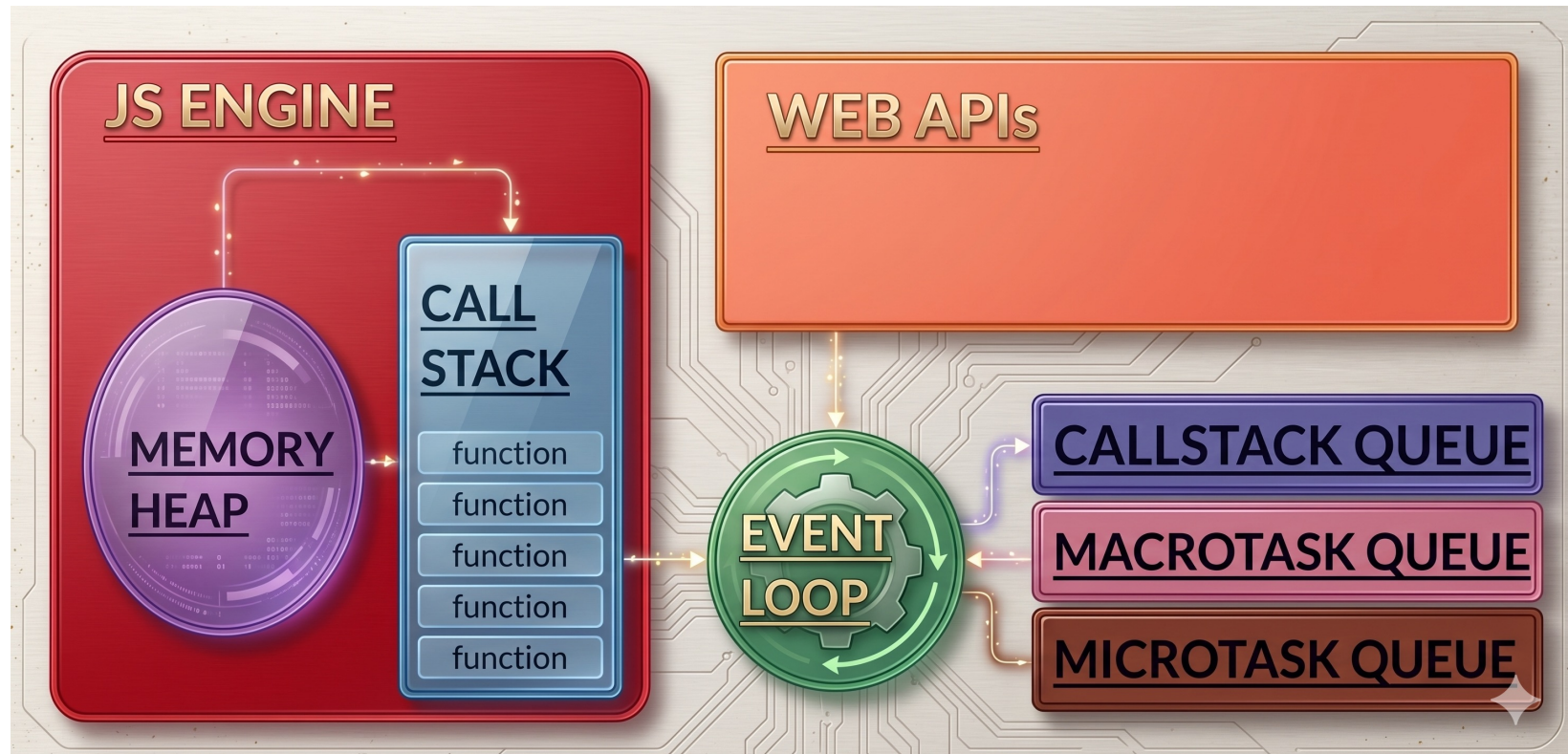
Dr. Wittawin Susutti

wittawin.sus@kmutt.ac.th

CSS234 WEB PROGRAMMING II

**LECTURE 06 JAVASCRIPT CONCURRENCY
& NETWORK FOUNDATIONS**

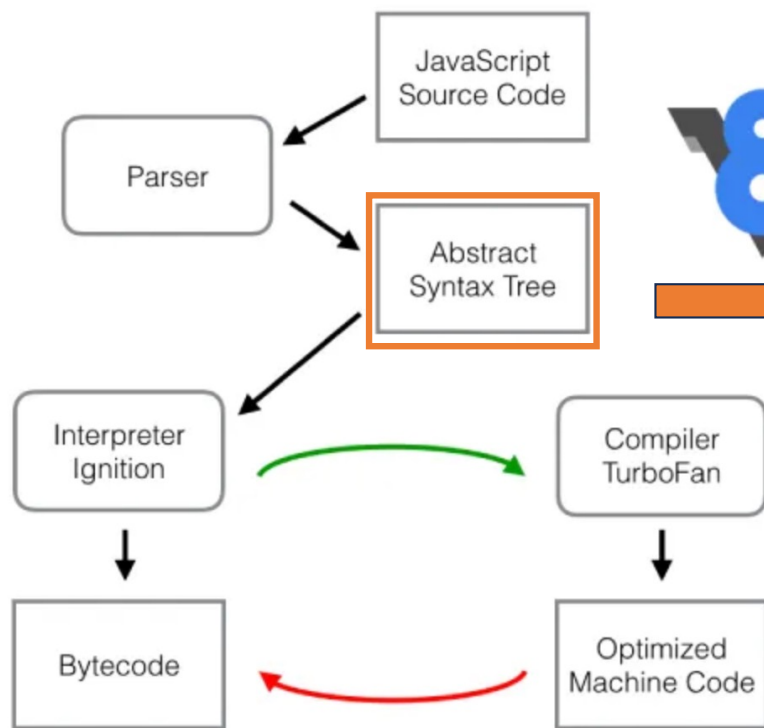
JavaScript Runtime Environment



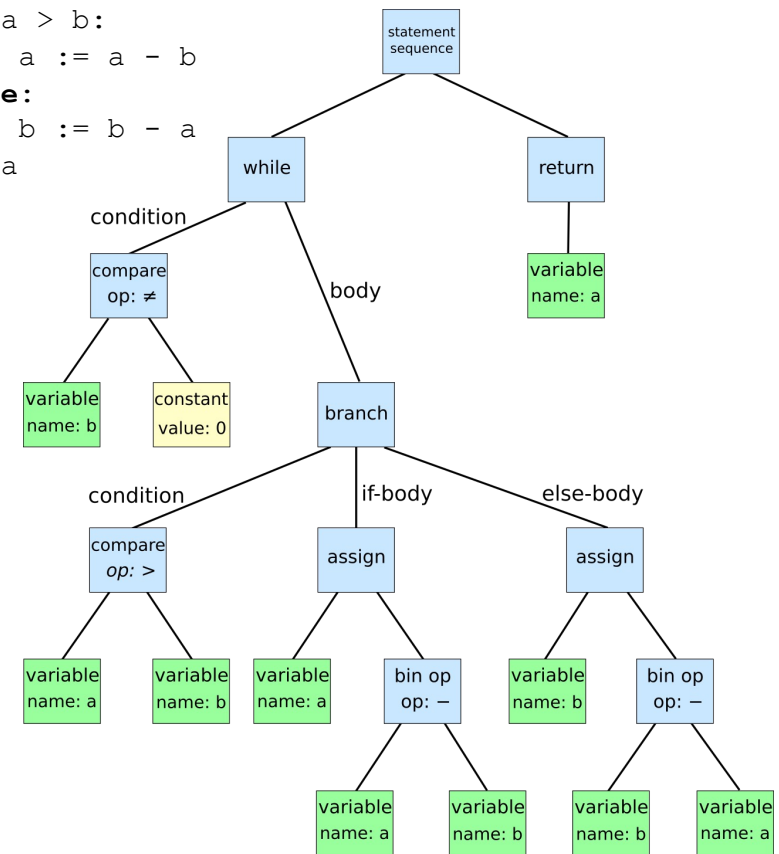
What is a JavaScript Engine?

- A program that executes JavaScript code by converting it into machine code that the CPU can understand.
- V8 Engine
 - Developed by Google.
 - Used in Google Chrome and Node.js.
- V8 is a Just-In-Time (JIT) compiler.
- A Just-In-Time (JIT) compiler improves runtime performance by translating bytecode or intermediate code into native machine code on-the-fly, during program execution.

The V8 Execution Pipeline



```
while b != 0:
    if a > b:
        a := a - b
    else:
        b := b - a
return a
```



@fhinkel

Phase 1: Parsing & Abstract Syntax Tree (AST)

- Scanner - Breaks code into 'Tokens' (keywords, operators, identifiers).
- Parser
 - Checks for syntax errors.
 - Builds an Abstract Syntax Tree (AST) — a tree representation of code structure.
- Efficiency - V8 uses 'Lazy Parsing' to skip functions not needed immediately.

Phase 2: Ignition (The Interpreter)

- Role - Converts the AST into Bytecode.
- Bytecode - Intermediate, platform-independent code.
 - Smaller and faster to generate than machine code.
- Immediate Execution: Starts running the code quickly while gathering optimization data.

Phase 3: TurboFan (The Optimizing Compiler)

- The Profiler - Monitors which parts of the code are 'Hot' (executed frequently).
- Optimization - TurboFan compiles 'Hot' bytecode into highly optimized Machine Code.
- Speculative Optimization - Makes assumptions based on previously seen types.

De-optimization

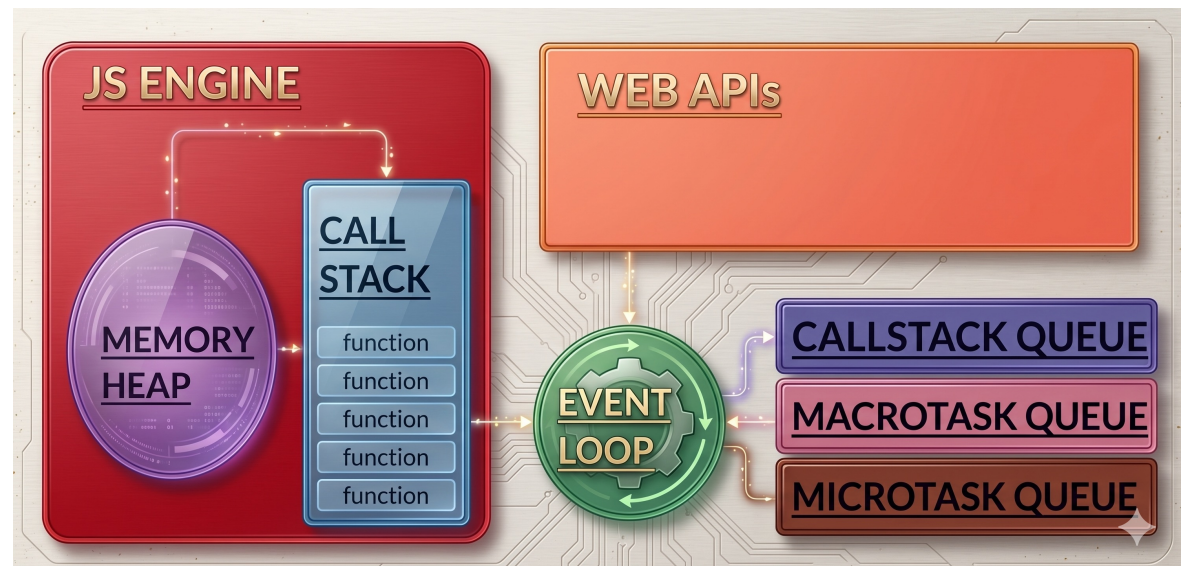
- Dynamic Typing Challenge - JavaScript types can change at runtime.
- The Reversal - If an assumption fails, TurboFan must De-optimize.
- Bailout - The engine discards optimized code and falls back to the Interpreter (Ignition).
- Performance Hit - Frequent de-optimization makes the code run much slower.

Memory Management & Garbage Collection

- Memory Heap - Where objects, strings, and closures are stored.
- Orinoco (The Garbage Collector)
 - Automatically reclaims memory by removing unreachable objects.
- Generational Collection - Young Generation (new) and Old Generation (long-lived).
- Parallel & Incremental - Collection in small chunks to avoid 'Stop-the-world' pauses.

The Single-Threaded Rule

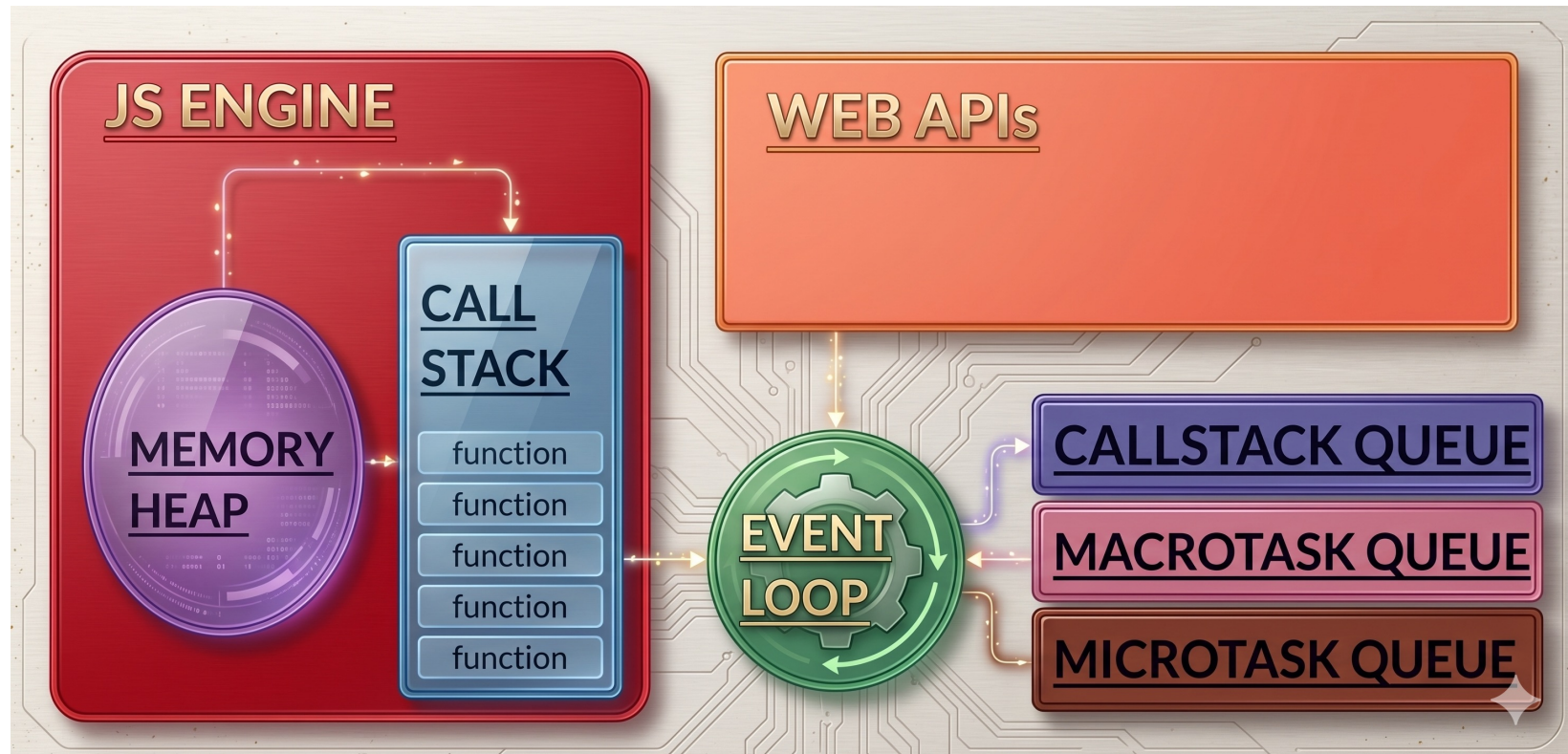
- The Single-Threaded Rule
 - JS can only do one thing at one time in the main thread.
- The Browser Environment (Web APIs)
 - The engine doesn't work alone.
 - The browser provides additional threads for DOM, Network (`fetch`), and Timers (`setTimeout`).



The Main Thread Bottleneck

- Synchronous Blocking
 - Long-running tasks prevent the browser from rendering the UI.
- Real-World Example
 - Parsing a 50MB CSV file synchronously on the main thread.
 - Applying a complex image filter (pixel manipulation) directly in the UI component.
- The Result - 'Jank' (Unresponsive UI), frozen animations, and frustrated users.

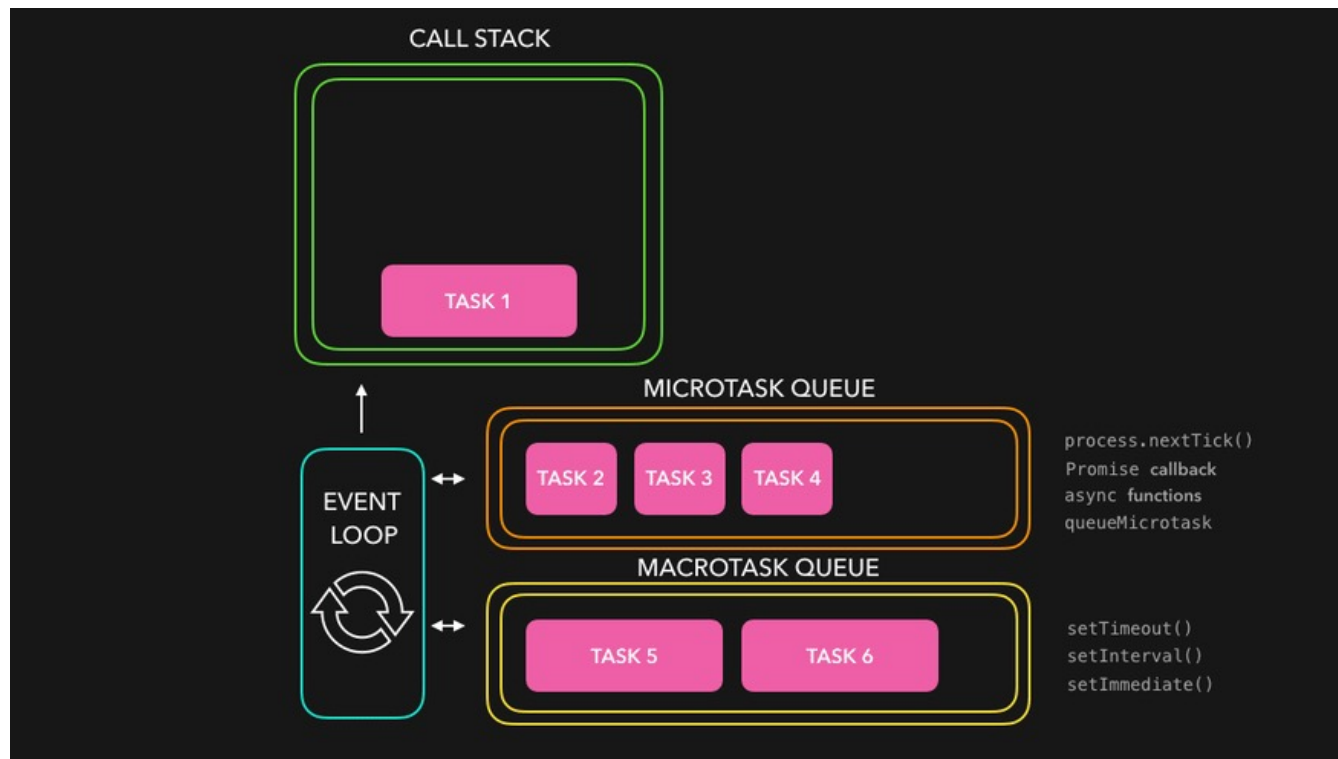
JavaScript Runtime Environment



The Event Loop & Task Queues

- The Event Loop checks if the Call Stack is empty.
- Macrotask Queue (Task Queue)
 - `setTimeout`, `setInterval`, UI rendering, I/O.
- Microtask Queue
 - Promises (`.then/.catch`), `queueMicrotask()`, `MutationObserver`.
- The Golden Rule of Execution
 - **The Event Loop will completely empty the Microtask Queue before processing the next single Macrotask.**

JavaScript Runtime Environment



Checkpoint - The Execution Order

Analyze this common question

```
console.log('A');  
  
setTimeout(() => console.log('B'), 0);  
  
Promise.resolve().then(() => console.log('C'));  
  
console.log('D');
```

The Evolution of Asynchronous JS

Phase 1: Callbacks

- Passing functions as arguments.
- Problem: Inversion of control, 'Callback Hell' (Pyramid of Doom).

Phase 2: Promises (ES6)

- Objects representing future values (Pending, Fulfilled, Rejected).
- Allows chaining (.then().then()).

Phase 3: Async/Await (ES8)

- Syntactic sugar over Promises.
- Makes asynchronous code look and behave like synchronous code.

Promise Combinators

- Handling Multiple Promises Concurrently
- `Promise.all([p1, p2])`
 - Resolves when all promises resolve. Rejects if any single promise rejects.
- `Promise.race([p1, p2])`
 - Resolves/Rejects as soon as the first promise settles.
- `Promise.allSettled([p1, p2])`
 - Waits for all to finish, regardless of success or failure.

Concurrent API Fetching

- Sequential vs. Concurrent Execution

- Bad (Sequential - 4 seconds total):

```
const user = await fetch('/api/user'); // 2s
const posts = await fetch('/api/posts'); // 2s
```

- Good (Concurrent - 2 seconds total):

```
const [user, posts] = await Promise.all([
  fetch('/api/user'), fetch('/api/posts')
]);
```

Async/Await & Error Handling

- The `try...catch` Block
 - Essential for handling rejected promises in `async` functions.
- Real-World Pattern

```
async function loadData() {  
  try {  
    const data = await fetchData();  
    renderUI(data);  
  } catch (error) {  
    showFallbackUI();  
  }  
}
```

The Fetch API

- Beyond Simple `GET` Requests
- The `Request` Object Configuration
 - `method`: `POST`, `PUT`, `DELETE`.
 - `headers`: Sending API Keys, Content-Type (e.g., `application/json`).
 - `body`: Serializing data (`JSON.stringify(data)`).
- CORS (Cross-Origin Resource Sharing)
 - A security mechanism by browsers.
 - Servers must explicitly allow requests from different origins.

Real-World Use Case - Cancelling Requests

- The Problem: Rapid typing in search boxes fires too many requests.

```
const controller = new AbortController();
```

```
fetch('/search?q=apple', { signal: controller.signal });
```

```
// Later, if user types something else before it finishes:
```

```
controller.abort(); // Cancels the network request instantly
```

Client-Side Storage Ecosystem

- `localStorage` / `sessionStorage`
 - Synchronous (Blocks the main thread).
 - Strictly for small string data (Tokens, UI themes).
- `IndexedDB`
 - Asynchronous, large quota, structured NoSQL database.
 - Perfect for JSON data and user state.
- `Cache API`
 - Asynchronous, specialized for Request/Response objects.
 - Perfect for App Shells (HTML, CSS, JS, Images).

Checkpoint - Storage & Network

- Scenario - Offline dashboard with real-time crypto prices.
- Question - Where should you store JSON data for immediate offline render?
- Answer - IndexedDB.
 - Asynchronous
 - Accessible from Workers
 - Handles structured JSON efficiently.